

CS & IT ENGINEERING

Programming in C



Strings
Lec- 01



By- Pankaj Sharma sir



TOPICS TO BE
COVERED

Srrings-l

string

A sequence of characters terminated by null character.

Ascii code - 0
'\0'

collection/group
of character.

a	b	c	\0
---	---	---	----

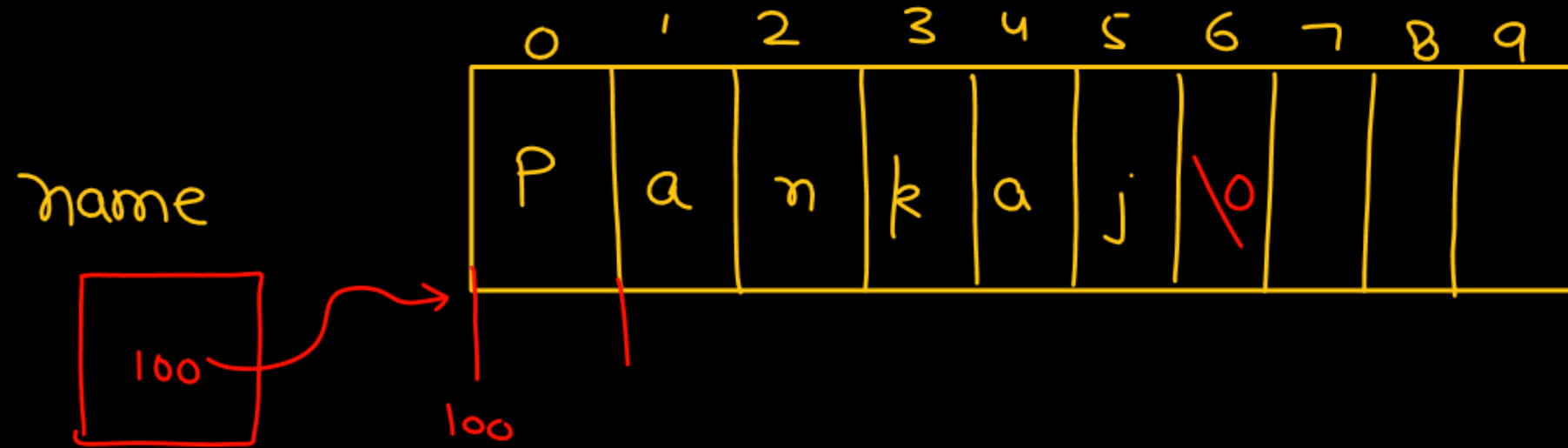
①

char name[10] = "Pankaj";

literal

" "

printf("%s", name)



②

```
char name[] = "Pankaj";
```

```
printf("%s", name);
```



③

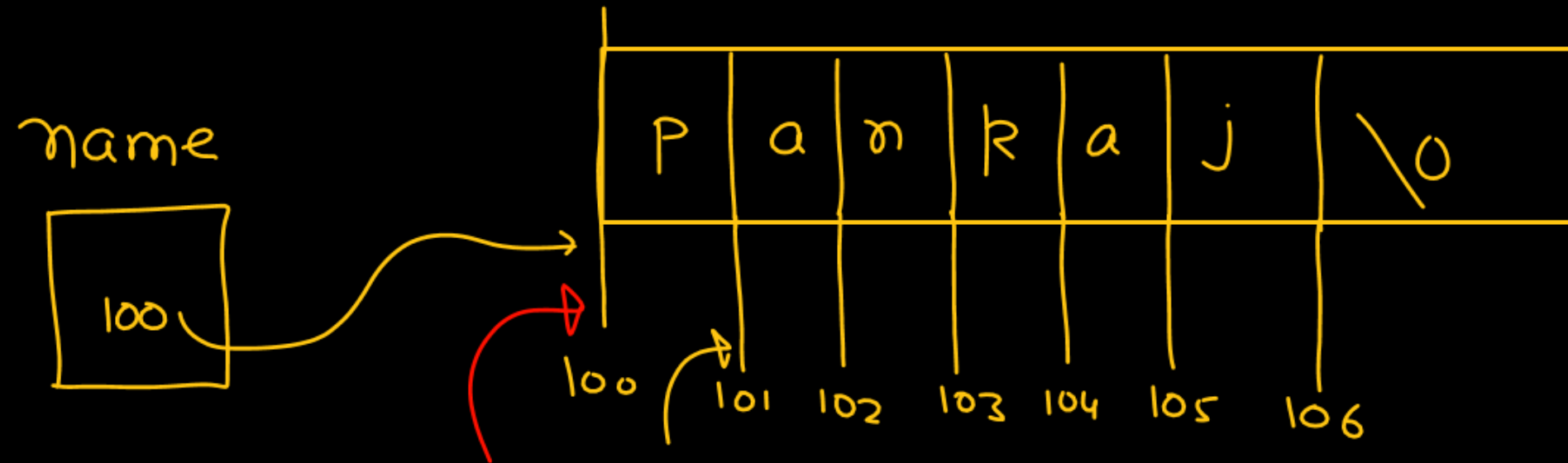
```
char name[] = { 'P', 'a', 'n', 'k', 'a', 'j' };
```

name

P	a	n	k	a	j	
---	---	---	---	---	---	--

```
printf("%s", name);
```

```
char name[10] = "Pankaj";
```



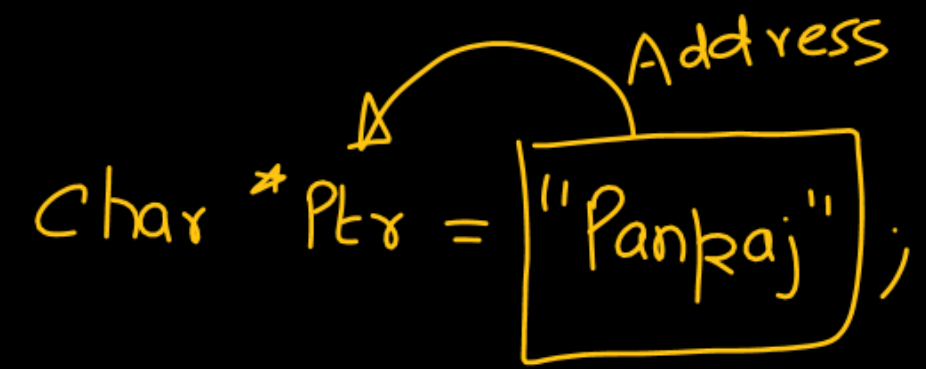
```
printf("%s", name);
```

Diagram showing the argument `name` in the `printf` function call. An arrow points from `name` to the address `&name[0]`, indicating that the address of the first element of the array is passed to the function.

```
printf("%s", name+1);
```

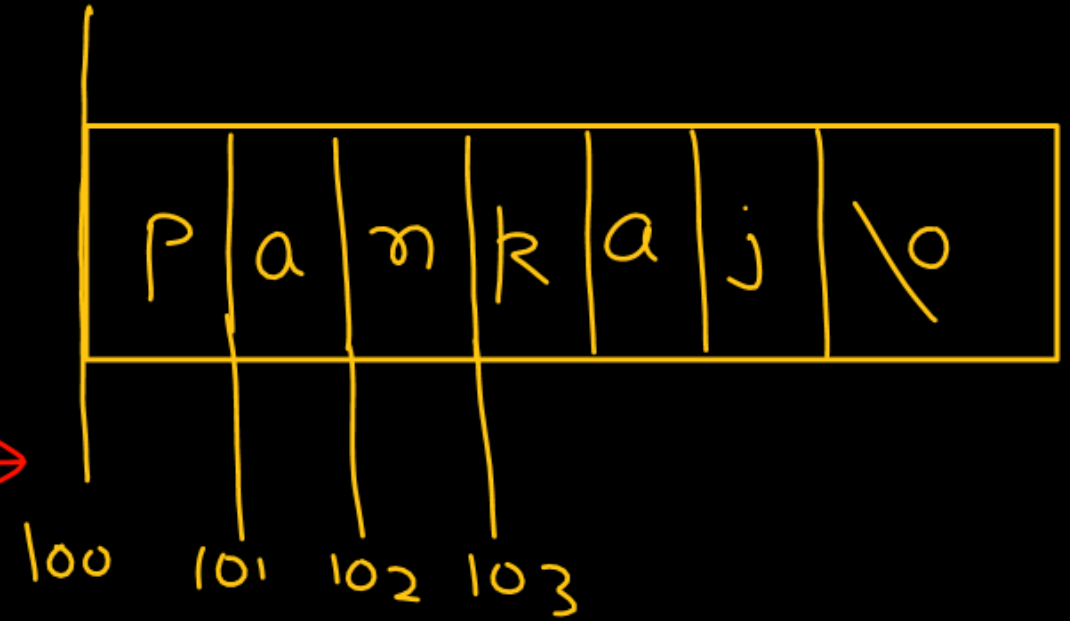
Diagram showing the argument `name+1` in the `printf` function call. An arrow points from `name+1` to the word `Address`, which is circled in red. Below the code, the output `ankaj` is shown, indicating that the first character 'P' is skipped.

Char *Ptr = "Pankaj";



printf("%s", Ptr);

Address




```
char arr[] = "Panpaj";
```

```
printf("/s", arr);
```

```
printf("/s", arr+1);
```

```
printf(arr);
```

```
printf(arr+1);
```

```
char *ptr = "neeraj";
```

```
printf("/s", ptr)
```

```
printf("/s", ptr+1);
```

```
printf(ptr);
```

```
printf(ptr+1);
```

char arr[] = "Pankaj";

arr++;

++arr;

--arr;

arr--;

arr = "neeraj";

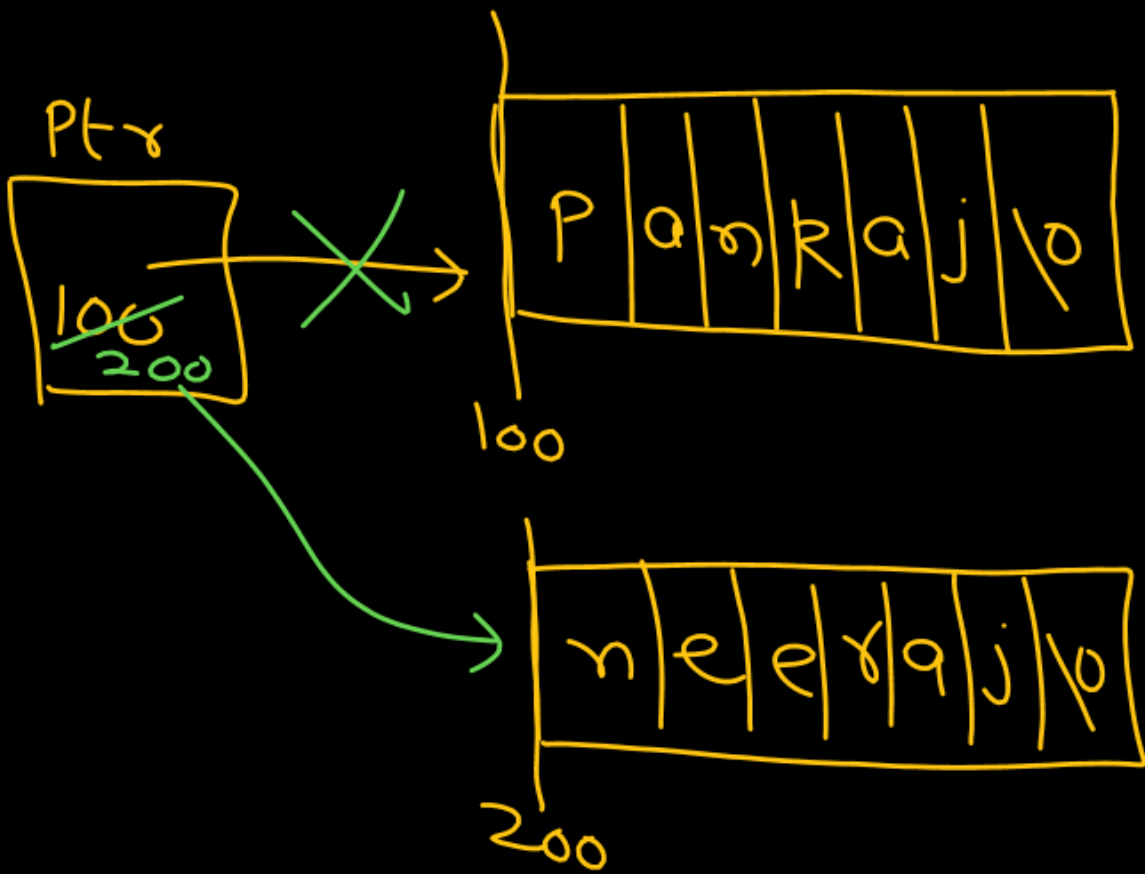
Errors

char *ptr = "Pankaj";

[++ptr; ✓
ptr++; ✓

ptr
↳ pointer variable

ptr = "neeraj";



```
int a[4] = {1, 2, 3, 4};
```

```
a[1] = 200; ✓
```

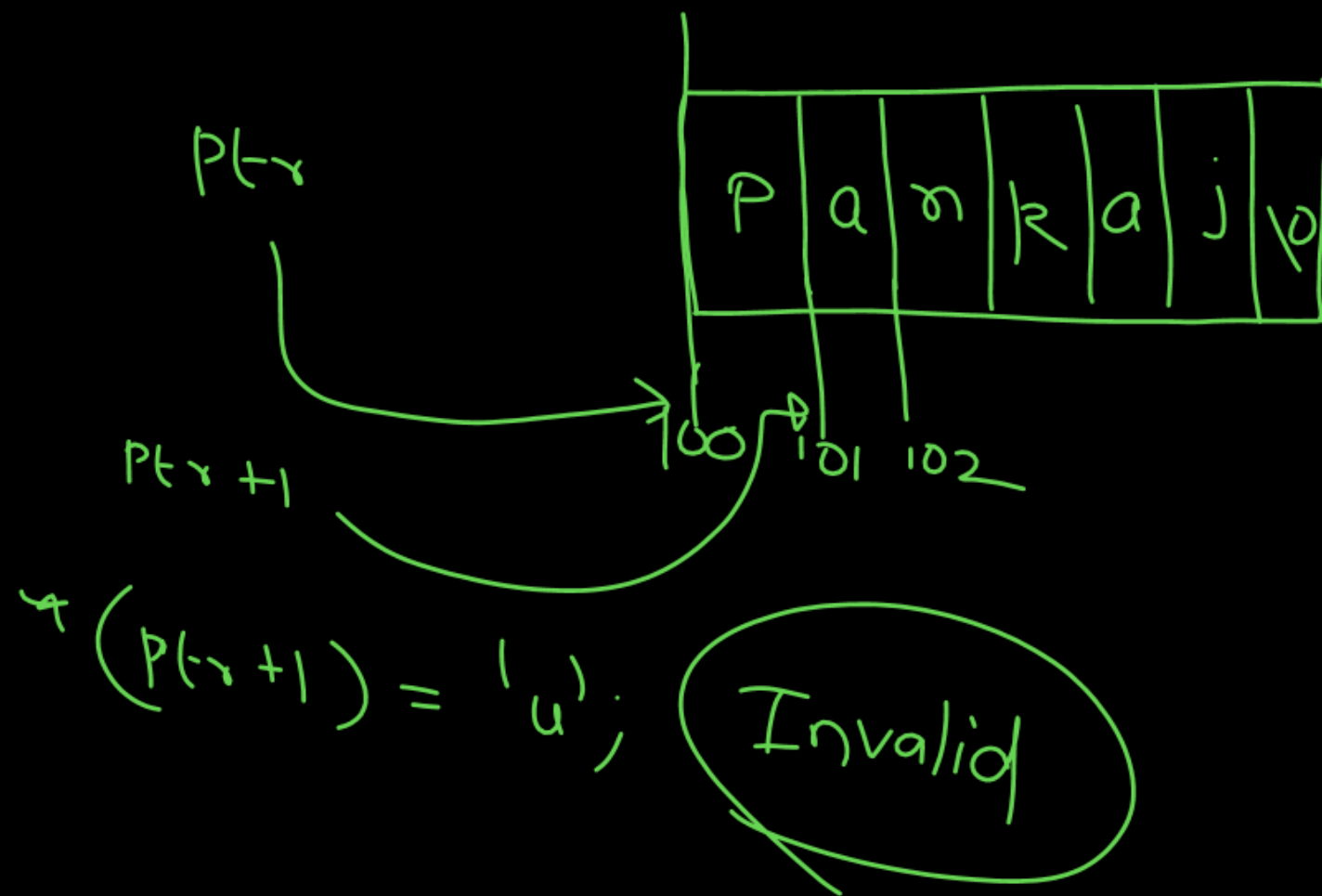
we can change
content of
array
i.e. ind. element

```
char arr[] = "Pankaj";  
arr[1] = 'u'; ✓
```

P	u	n	k	a	j	\0
---	---	---	---	---	---	----

```
char* ptr = "Pankaj";
```

Read only
area



`printf("Hello");`

①

Read only
area

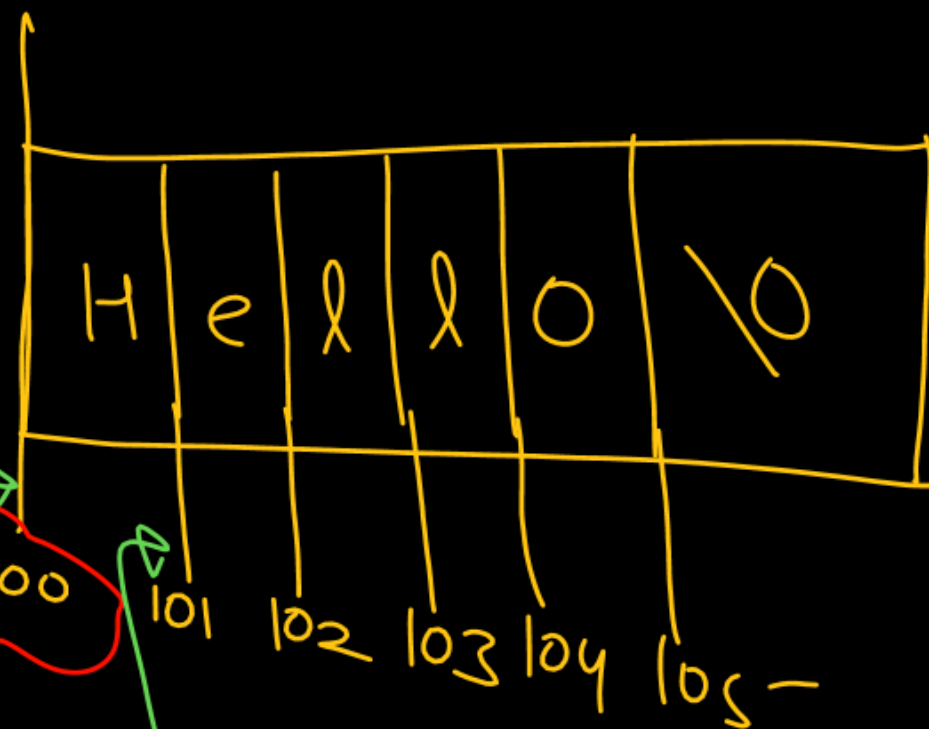
"Hello" $\Rightarrow$ Add. of 'H'

`printf("Hello")` $\Rightarrow$ `printf(Add of 'H')`

"Hello" $\Rightarrow$ Add. of 'H'

"Hello" + 1 $\Rightarrow$ Add. of 'e'

`printf("Hello" + 1)` $\Rightarrow$ `ello`



char name[] = "Pankaj"; char *ptr = "name"

OR

void main() {

int a = {1}; valid ✓

char name[] = {"Pankaj"};

Ram going to is Aggra

int a[] = { }

"Hello" $\Rightarrow$ add. of 'H'

"Hello" + 1 $\Rightarrow$ add. of 'e'

*("Hello" + 1) $\Rightarrow$ 'e'

printf("%c", "Hello"[1]);
e


```
void main() {  
    if ( "Pankaj"[2] == "neeraj"[0] )  
        printf("Pankaj" "sir"); ✓  
    else  
        printf("Gate" "Wallah" "cs");  
}
```

Pankaj sir

char arr[] = "GATE";

0	1	2	3	
G	A	T	E	\0

Diagram illustrating the memory layout of the string "GATE". The string is stored in an array of characters. The indices 0, 1, 2, and 3 correspond to the characters 'G', 'A', 'T', and 'E' respectively. The null terminator '\0' is at index 4. Arrows indicate the starting points for the printf statements: arr+1 points to index 1, arr+3 points to index 3, and arr+4 points to index 4.

① printf(arr+1); ATE

② printf(arr+3-1); TE

③ printf("/d", 'B'-'A');

③ printf("/s", arr+3[arr]-arr[1]);

arr+'E'-'A'

arr+4

Ascii
code Ascii
code code

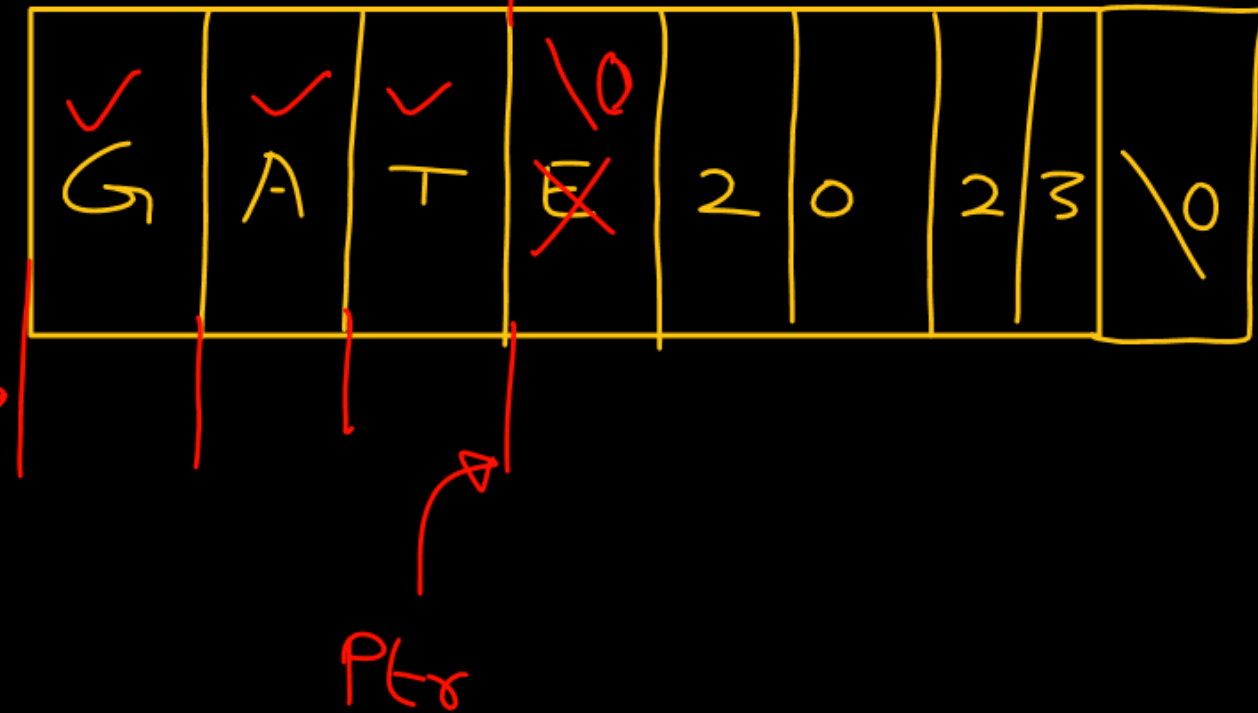
```
char arr[] = "GATE2023";
```

```
char *ptr;
```

```
ptr = arr + 3; ✓
```

```
*ptr = '\0';
```

```
printf("%s", arr);
```



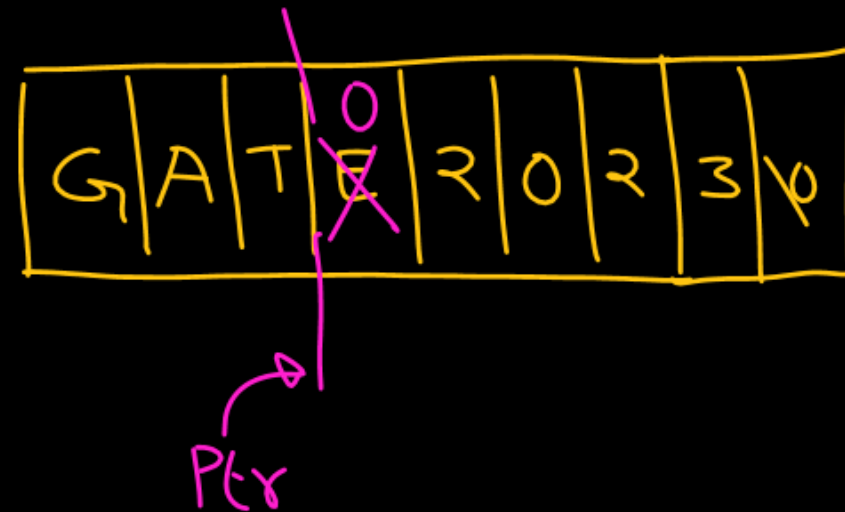
```
char arr[] = "GATE2023";
```

```
char *ptr;
```

```
ptr = arr + 3;
```

```
*ptr = '0';
```

```
printf("/s", arr);
```



```
char arr[] = "GATE2023";
```

```
char *ptr;
```

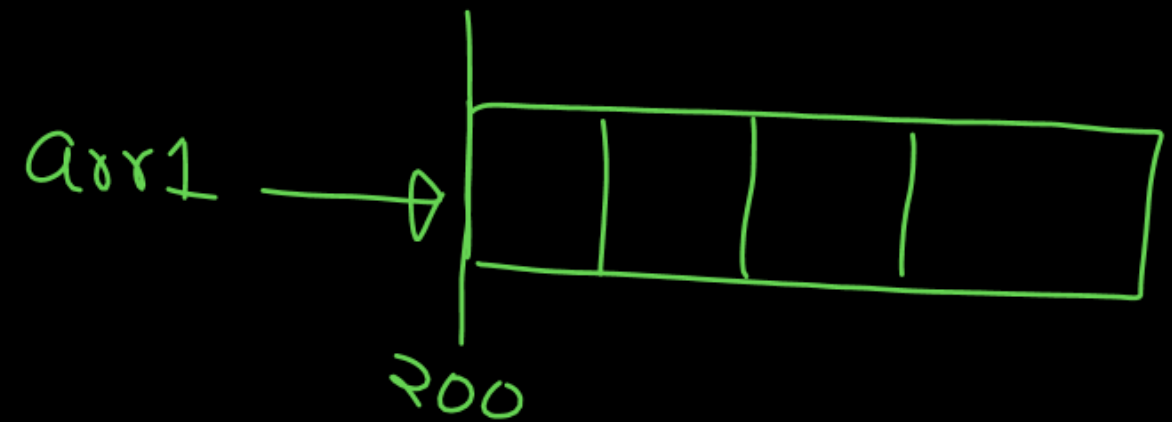
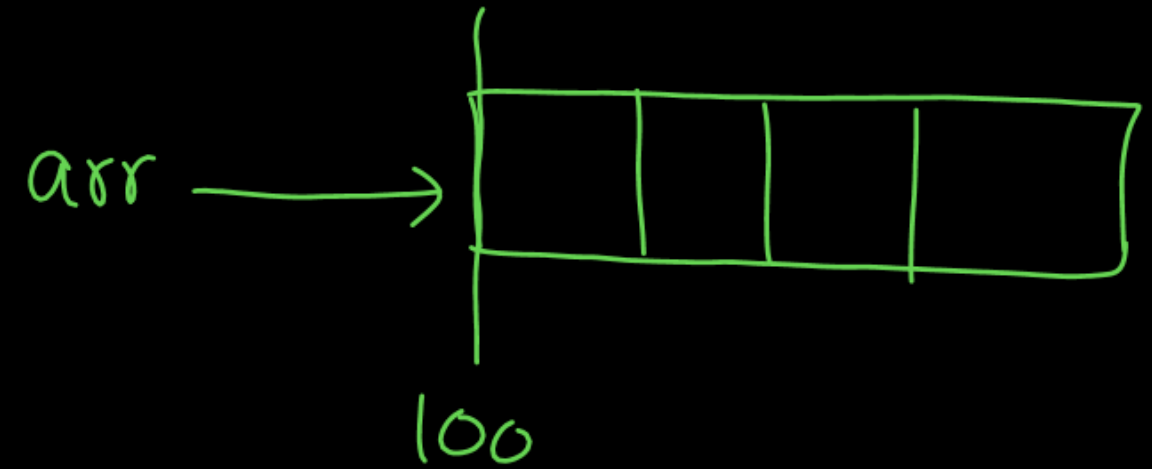
```
ptr = arr + 3;
```

```
*ptr = 0;
```

```
printf("/s", arr); GAT
```



`int arr[4];`
`int arr1[4];` } 2 diff. arrays



```
char arr1[] = "Pankaj";
```

```
char arr2[] = "Pankaj";
```

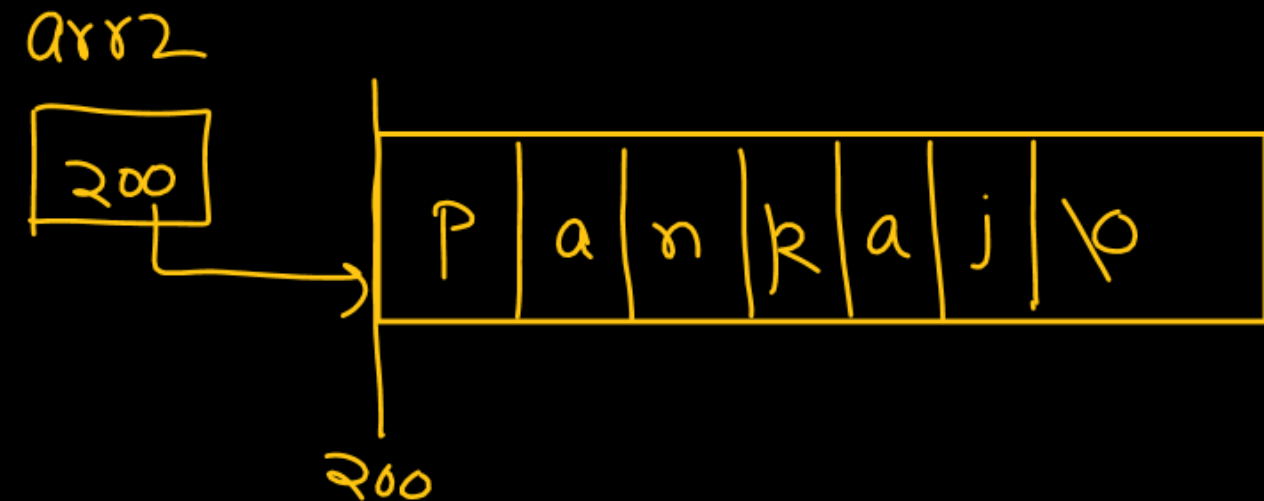
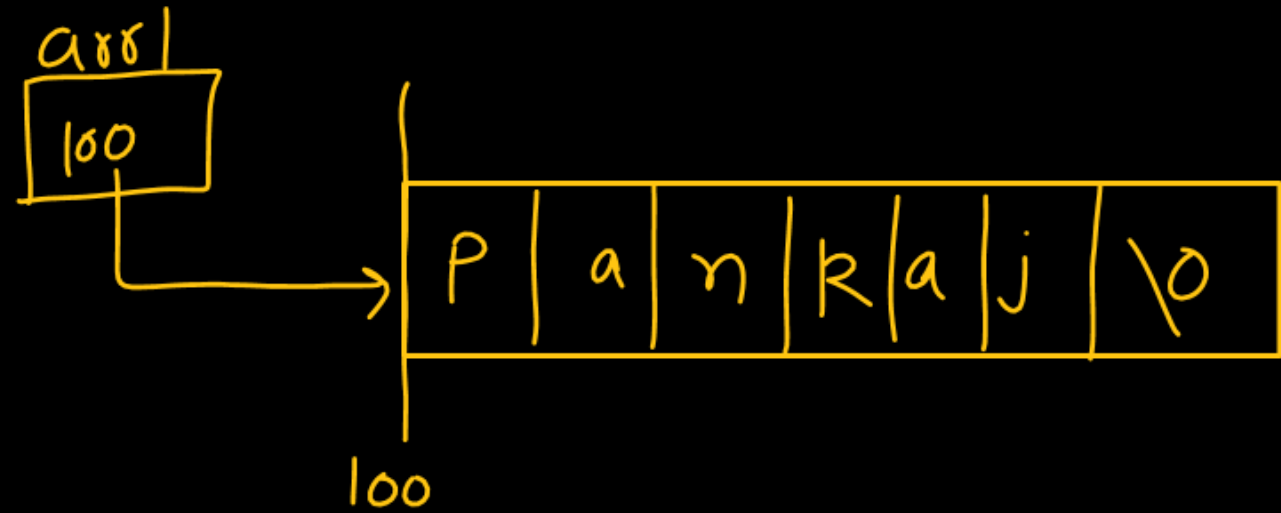
```
if ( arr1 == arr2 )
```

```
    printf("same");
```

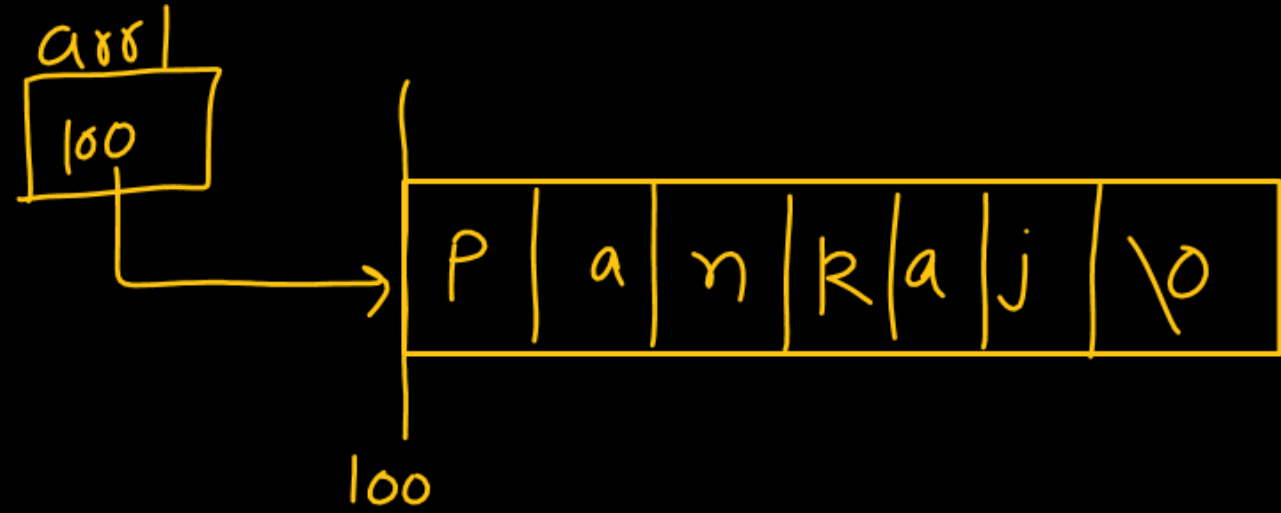
```
else
```

```
    printf("different");
```

O/P : different




```
char arr1[] = "Pankaj";
```



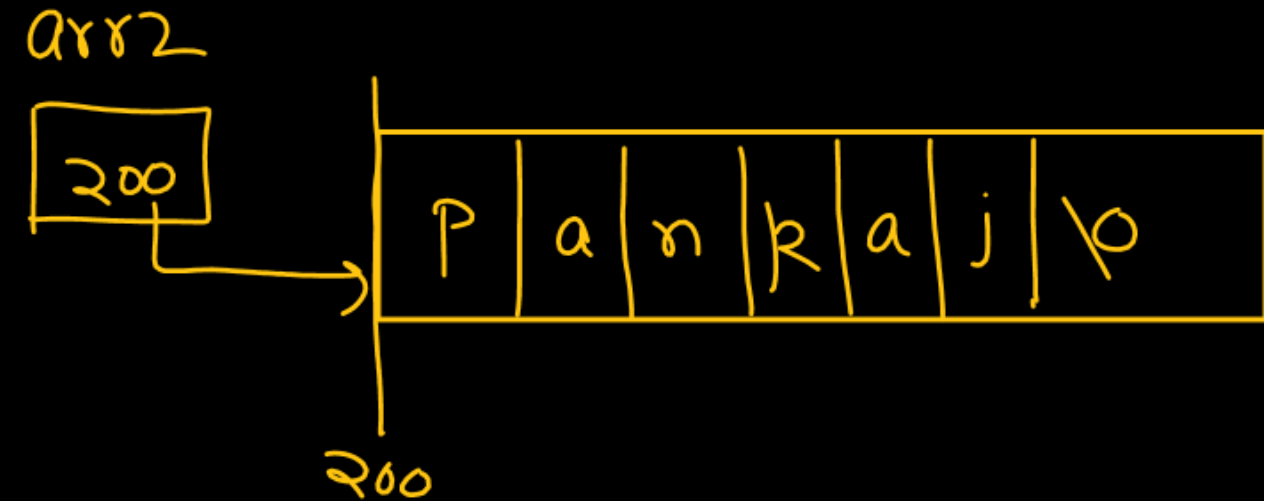
```
char arr2[] = "Pankaj";
```

```
if (*arr1 == *arr2)
```

```
    printf("same");
```

```
else
```

```
    printf("different");
```



O/p : same

char *p = "Pankaj";

Read only
memory area

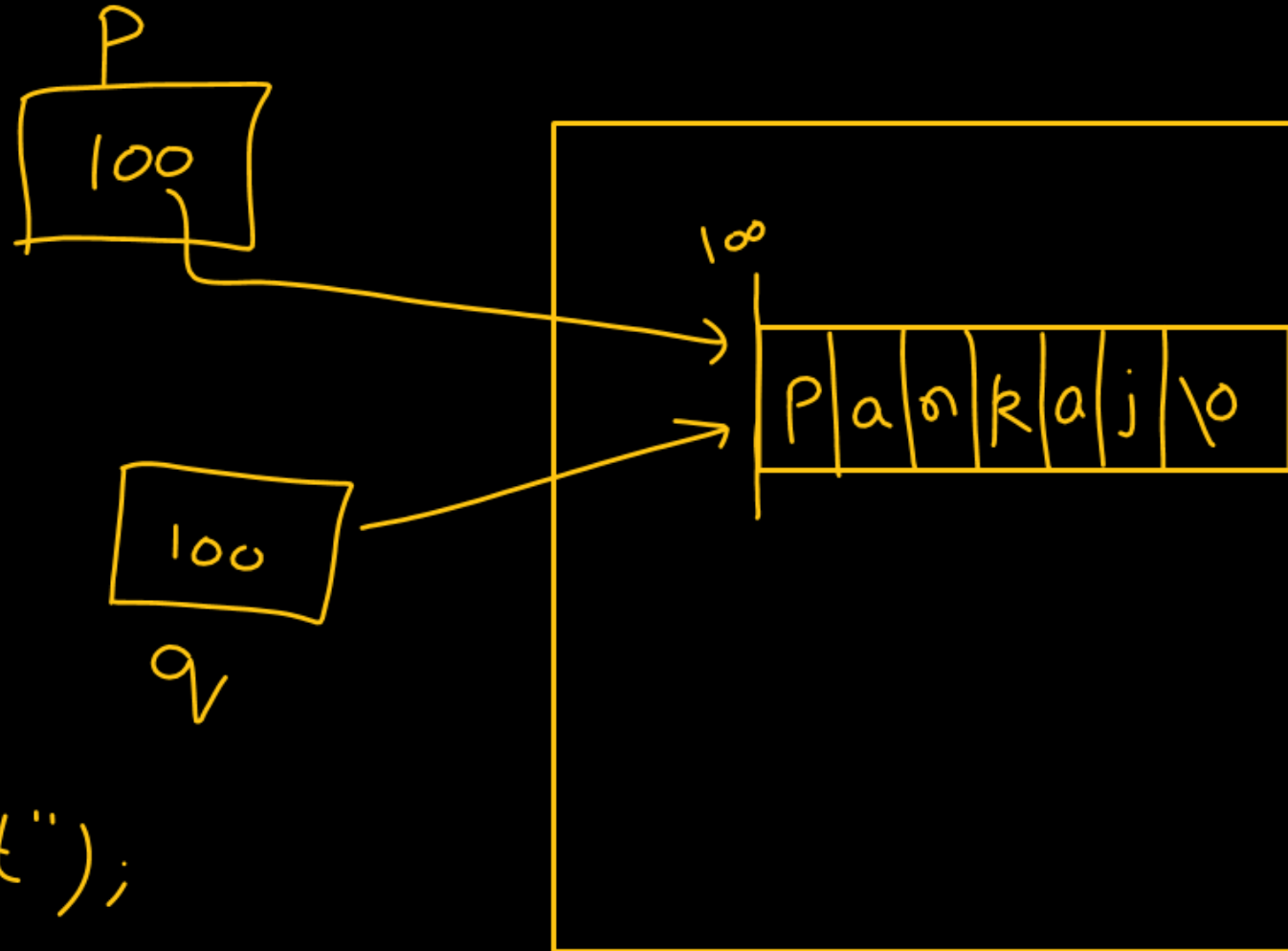
char *q = "Pankaj";

if (p == q)

printf("same");

else

printf("different");



Collection of string

1 string → a) char arr

char arr[] = "Parikaj";

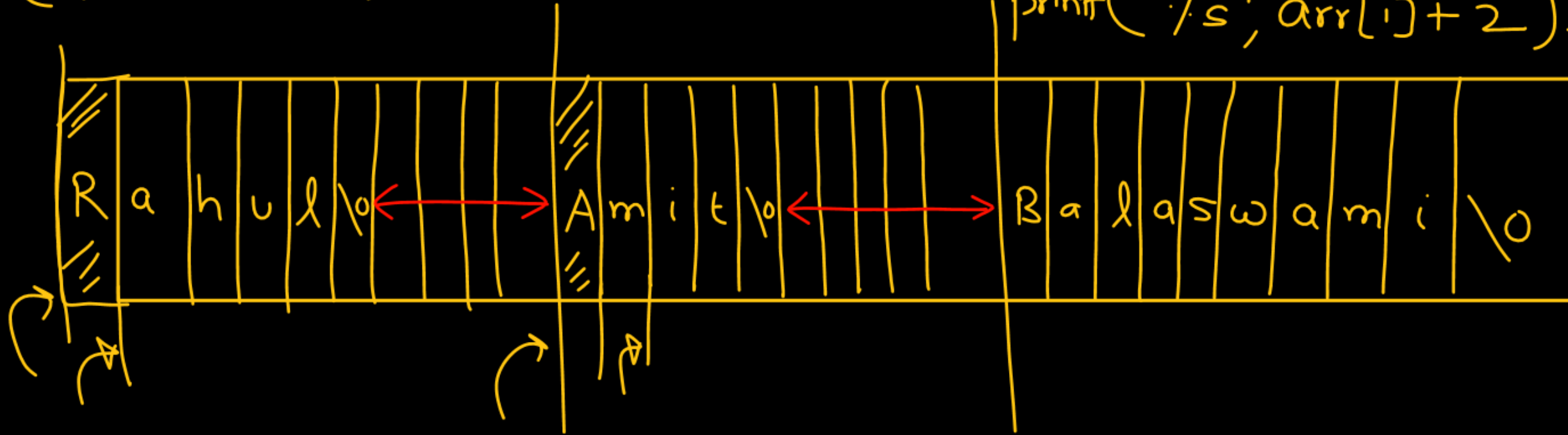
→ b) Pointer to a char

char *ptr = "neeraj";

`char arr[3][10] = {"Rahul", "Amit", "Balaswami"};`

`printf("%s", arr[1])` $\Rightarrow$ Amit

`printf("%s", arr[1] + 2)` $\Rightarrow$ it



`printf("%c", arr[0][1])` $\Rightarrow$ a

`printf("%s", arr[0]);`
`printf("%s", arr[0] + 1);` ahul

```
char *p = "neeraj";
```

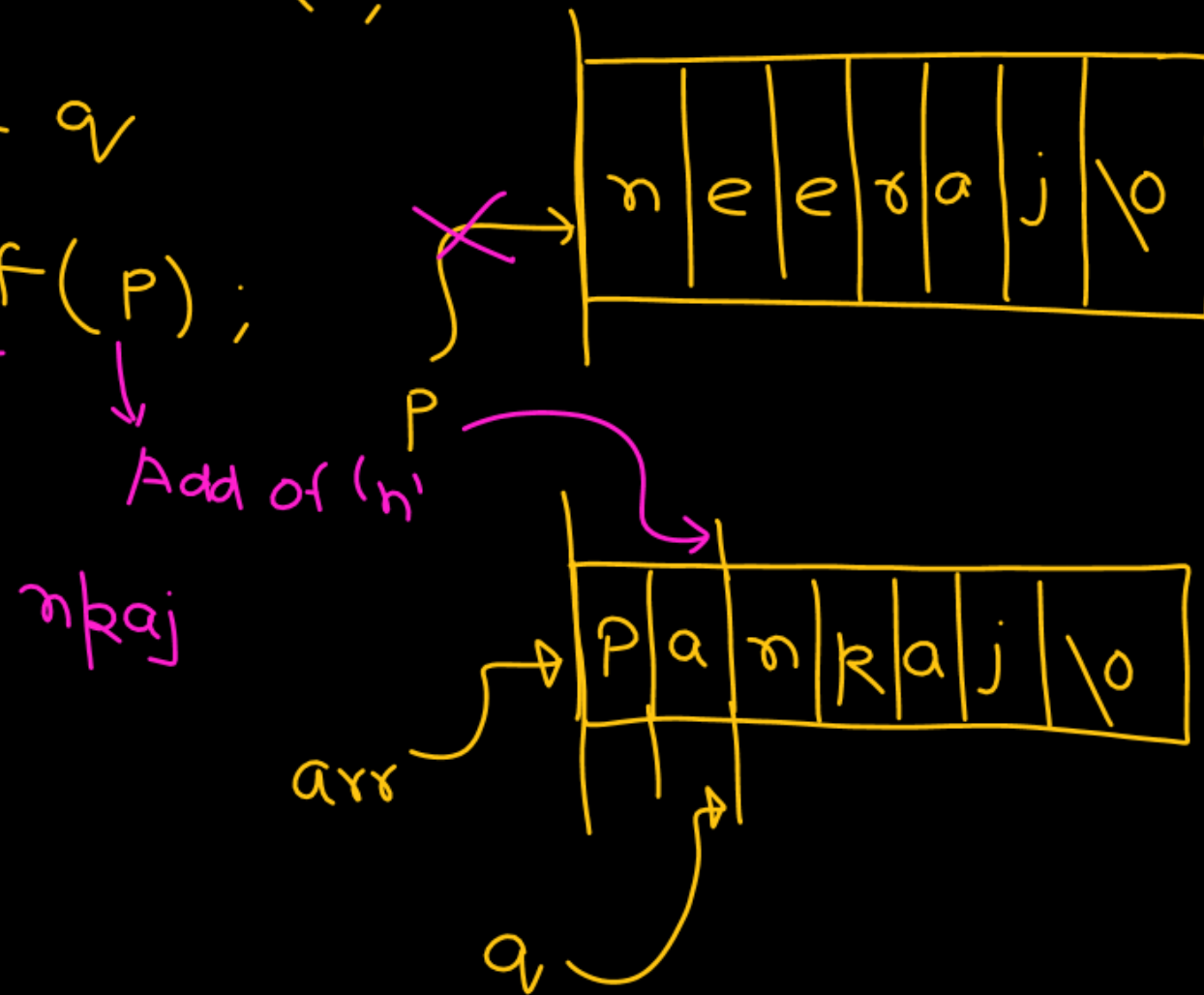
```
char arr[] = "Pankaj";
```

```
char *q;
```

```
q = arr + 2;
```

```
p = q
```

```
printf(p);
```



```
char arr[] = "Hello";
```

```
char *ptr = "Pankaj";
```

```
arr = ptr + 3;
```

```
printf("/s", arr + 1);
```

Can not be Lvalue
for any assignment

```
char a[3][10] = {"Amit", "Rahul", "Balaswami"};
```

```
char *ptr = "Pankaj";
```

```
a[0] = ptr + 2; error
```

```
printf("%c", a[0][2]);
```